

BiblioTech

Système Intégré de Gestion de Bibliothèque — SaaS Multi-tenant

Dossier de Projet

Livrable final — Formation CDA

Auteur :	Grégory Sergent
Formation :	Concepteur Développeur d'Applications — CDA
Organisme :	IPSSI Grande École d'Informatique
Période :	Janvier → Juillet 2026
Version :	1.0 — Juillet 2026

SOMMAIRE

1.	Contexte et problématique	3
2.	Objectifs du projet	4
3.	Méthodologie et organisation	5
4.	Conception UI/UX	6
5.	Modélisation de la base de données	8
6.	Architecture technique	11
7.	Développement réalisé	15
8.	Fonctionnalités avancées	19
9.	Sécurité	21
10.	Tests	25
11.	Infrastructure & CI/CD	29
12.	Conformité RGPD	31
13.	Bilan et perspectives	32

1. Contexte et Problématique

1.1 Constat

Le secteur des bibliothèques municipales et associatives en France fait face à un défi de modernisation. Si les grandes structures disposent de progiciels complexes (SIGB commerciaux), de nombreuses petites et moyennes bibliothèques utilisent encore des méthodes manuelles : registres papier, fichiers Excel non partagés, suivi des prêts par téléphone.

Cette situation entraîne des pertes d'informations, un manque de visibilité sur l'état du stock en temps réel, et une charge administrative importante pour les bibliothécaires.

1.2 Commanditaire fictif

■ ■ Le commanditaire, la structure et les personnes mentionnées sont entièrement fictifs. Ils ont été créés dans le cadre d'un exercice pédagogique.

Champ	Information
Nom	L'IPSSlthèque (fictif)
Type de structure	Bibliothèque associative — Aubusson, Creuse (23)
Contact projet	Mme Isabelle Perrin, directrice (fictif)
Situation actuelle	Gestion sur registres papier et tableur Excel non partagé
Besoin exprimé	Outil numérique simple, centralisé, pour catalogue, adhérents et prêts

1.3 Solution proposée : BiblioTech

BiblioTech est une plateforme SaaS multi-tenant conçue pour simplifier la transition numérique des bibliothèques associatives et municipales. Elle s'adresse au personnel de la bibliothèque (bibliothécaires, admins) ainsi qu'aux adhérents en lecture seule.

Périmètre fonctionnel :

- Authentification sécurisée avec gestion des rôles (RBAC à 4 niveaux)
- Gestion du catalogue : ajout par ISBN via Google Books API, CRUD complet
- Gestion des adhérents et des comptes utilisateurs
- Module de prêts/retours : création, retour, détection automatique des retards
- Tableau de bord avec statistiques (prêts en cours, retards, fonds)
- Demandes de migration entre bibliothèques
- Gestion des CGU versionnées avec préavis de 15 jours

2. Objectifs du Projet

2.1 Objectifs SMART

Objectif	Description	Critère de succès
Digitaliser 100% des flux	Supprimer les registres papier au profit d'une BDD centralisée	0 prêt/retour traité hors de l'application
Automatiser l'inventaire	Réduire le temps de saisie d'un livre via l'API Google Books	Ajout en < 5 secondes par ISBN
Optimiser le suivi des retards	Identifier instantanément les livres non rendus	Liste des retards disponible en 1 clic
Conformité RGPD	Registre de traitement, droit à l'oubli, Argon2id	Implémenté avant mise en production
Architecture SaaS	Plateforme multi-tenant isolant chaque bibliothèque	Isolation totale par bibliotheque_id sur tous les endpoints

2.2 Calendrier du projet

Jalon	Mois	Livrable
Jalon 1	Janvier 2026	Cahier des charges fonctionnel (CDCF)
Jalon 2	Février 2026	Méthodologie & conception UI/UX
Jalon 3	Mars 2026	Modélisation BD (MCD, MLD, MPD PostgreSQL)
Jalon 4	Avril 2026	Conception UML & architecture technique
Jalon 5	Mai/Juin 2026	Développement complet, sécurité, tests (version bêta)
Jalon 6	Juillet 2026	Audit sécurité, CI/CD, déploiement, livrable final

3. Méthodologie et Organisation

3.1 Méthode de gestion : Agile Scrum adapté en solo

Le projet a été conduit selon une approche **Agile Scrum adaptée au contexte d'un développeur unique**. Chaque jalon correspond à un sprint mensuel avec des objectifs définis en amont.

Pratique	Description
Sprint mensuel	Un sprint par jalon (janvier → juillet 2026)
Sprint planning	Définition des objectifs du jalon + découpage en tâches (Trello)
Revue hebdomadaire	Contrôle de l'avancement, mise à jour du backlog
Rétrospective mensuelle	Bilan des difficultés et ajustements
Trello Kanban	Backlog → À faire → En cours → En revue → Terminé

3.2 Gestion du code source (Git)

Branche	Usage
`main`	Versions stables (releases, jalons)
`develop`	Intégration continue — branche principale de travail
`feature/*`	Nouvelles fonctionnalités ou correctifs (PR vers develop)

Conventions : commits Conventional Commits (**feat**:, **fix**:, **chore**:, **docs**:) sur toutes les branches. Merge avec PR auto-revue systématique.

4. Conception UI/UX

4.1 Sitemap

L'application BiblioTech est organisée autour de 8 sections principales, accessibles depuis la sidebar de navigation :

Section	Rôles	Fonctionnalités
Connexion	Tous	Formulaire JWT, gestion premier login et CGU
Tableau de bord	Staff + adhérent	KPIs, prêts en retard, statistiques
Catalogue	Tous	Liste des livres, ajout ISBN, CRUD exemplaires
Prêts / Retours	Staff	Création prêt, retour, prêts perdus, retards
Adhérents	Admin +	CRUD utilisateurs, suspensions, migrations
Bibliothèque	Admin +	Config bibliothèque, durée de prêt, activation
Demandes de migration	Tous	Soumettre / valider / refuser une migration
Mon Compte	Tous	Profil, sécurité, RGPD, CGU, gestion CGU (super_admin)

4.2 Charte graphique

Rôle	Valeur	Utilisation
Primaire	#1E3A8A — Bleu nuit	Sidebar, titres, boutons principaux
Secondaire	#60A5FA — Bleu clair	Accents, liens, badges
Validation	#16A34A — Vert	Actions de confirmation, statuts OK
Neutre clair	#F3F4F6 — Gris clair	Fond de page, lignes alternées tableaux
Neutre foncé	#374151 — Gris foncé	Texte courant

Typographie :

Rôle	Police	Taille
Titres H1/H2	Sora — Bold	28px / 20px
Texte courant	DM Sans — Regular	14px
Labels / tableaux	DM Sans — Medium	13px
Codes ISBN / techniques	JetBrains Mono	13px

4.3 Principes UX

- **Scan ISBN rapide** : ajout d'un livre en moins de 5 secondes via la Google Books API

- **Feedback visuel** : messages de confirmation/erreur sur chaque action critique
- **Navigation filtrée par rôle** : la sidebar n'affiche que les sections accessibles
- **Mobile-first partiel** : layout responsive (sidebar ↔ drawer mobile) adapté à un usage desktop professionnel
- **Listes paginées** : chargement optimisé même avec un grand catalogue

5. Modélisation de la Base de Données

5.1 Démarche MERISE

La modélisation suit la méthode **MERISE** en trois niveaux progressifs : MCD (Modèle Conceptuel des Données), MLD (Modèle Logique) et MPD (Modèle Physique pour PostgreSQL). Cette démarche garantit une conception rigoureuse, indépendante du SGBD dans un premier temps.

5.2 Entités principales

Entité	Table PostgreSQL	Champs clés
Bibliothèque	bibliothèques	nom, adresse, email_contact, actif, duret_pret_jours
Utilisateur	utilisateurs	nom, prenom, email, mot_de_passe (Argon2id), role, bibliotheque_id, must_change_password, login_attempts, locked_until, cgu_accepted_version
Livre	livres	isbn, titre, auteur, editeur, annee_publication, genre, resume, couverture_url
Exemplaire	exemplaires	livre_id, bibliotheque_id, statut (disponible/emprunte/perdu/retire)
Pret	prets	exemplaire_id, adherent_id, bibliotheque_id, date_emprunt, date_retour_prevue, statut, etat_depart, etat_retour
CguVersion	cgu_versions	version, contenu (TEXT brut), date_effet, date_publication, publie_par
DemandeMigration	demandes_migration	adherent_id, bibliotheque_source_id, bibliotheque_cible_id, statut
RefreshToken	refresh_tokens	token, valid_until, utilisateur_id

5.3 Relations et cardinalités

Relation	Cardinalité	Description
bibliothèques → utilisateurs	(1,1) – (0,n)	Une bibliothèque a plusieurs utilisateurs
bibliothèques → exemplaires	(1,1) – (0,n)	Une bibliothèque possède plusieurs exemplaires
livres → exemplaires	(1,1) – (0,n)	Un livre peut avoir plusieurs exemplaires
exemplaires → prets	(1,1) – (0,n)	Un exemplaire peut avoir plusieurs prêts successifs
utilisateurs → prets	(1,1) – (0,n)	Un adhérent peut avoir plusieurs prêts
utilisateurs → demandes_migration	(1,1) – (0,n)	Un adhérent peut soumettre plusieurs demandes

5.4 Justifications de modélisation

- **Catalogue global + exemplaires séparés** : évite la duplication des métadonnées. Un livre est référencé une fois dans `livres`, le stock physique est géré dans `exemplaires` par bibliothèque.
- **bibliotheque_id dans prêts** : redondant mais conservé pour simplifier et optimiser les requêtes de reporting par bibliothèque sans jointure supplémentaire.
- **Rôle unique dans utilisateurs** : une seule table d'authentification. Le filtre par bibliotheque_id garantit l'isolation des données (principe de moindre privilège, ANSSI).
- **ON DELETE RESTRICT** : sur les FK critiques, empêche la suppression accidentelle d'une bibliothèque ou d'un exemplaire tant qu'il existe des prêts rattachés.
- **Normalisation 3NF** : aucune dépendance transitive. Chaque attribut dépend uniquement de la clé primaire de sa table.

5.5 Script SQL — Schéma (extrait)

```
CREATE TABLE utilisateurs (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    mot_de_passe VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN
        ('super_admin', 'admin', 'bibliothecaire', 'adherent')),
    bibliotheque_id INTEGER REFERENCES bibliotheques(id) ON DELETE SET NULL,
    must_change_password BOOLEAN DEFAULT TRUE,
    login_attempts INTEGER DEFAULT 0,
    locked_until TIMESTAMP NULL
);

CREATE TABLE prêts (
    id SERIAL PRIMARY KEY,
    exemplaire_id INTEGER NOT NULL REFERENCES exemplaires(id),
    adherent_id INTEGER NOT NULL REFERENCES utilisateurs(id),
    bibliotheque_id INTEGER NOT NULL REFERENCES bibliotheques(id),
    date_emprunt DATE NOT NULL DEFAULT CURRENT_DATE,
    date_retour_prevue DATE NOT NULL,
    date_retour_effective DATE,
    statut VARCHAR(20) NOT NULL DEFAULT 'en_cours'
    CHECK (statut IN ('en_cours', 'rendu', 'en_retard', 'perdu'))
);
```

6. Architecture Technique

6.1 Stack technique

Couche	Technologie	Version	Rôle
Backend	Symfony	7.3	Framework PHP — API REST MVC
Langage backend	PHP	8.3	Langage serveur
ORM	Doctrine	3.x	Mapping objet-relationnel + migrations
Auth	LexikJWTAuthenticationBundle	3.x	JWT RS256 stateless
Frontend	React	19.2	SPA — Single Page Application
Build tool	Vite	8.x	Bundler ultra-rapide pour React
Langage frontend	TypeScript	6.x	Typage statique
CSS	Tailwind CSS	4.x	Utility-first CSS framework
HTTP client	Axios	latest	Appels API + intercepteurs JWT
Base de données	PostgreSQL	16	SGBD relationnel
Conteneurisation	Docker / Compose	24+	Environnement reproductible
CI/CD	GitHub Actions	N/A	Tests + build Docker automatiques
Production DB	Supabase	cloud	PostgreSQL cloud managé

6.2 Architecture n-tiers

```
TIER 1 – Client (navigateur web)
  React 19 SPA + Tailwind CSS
  Appels HTTP/HTTPS → Authorization: Bearer
    ■
    ▼ HTTPS
TIER 2 – Serveur applicatif
  Symfony 7 + PHP 8.3 (FPM + Nginx)
  LexikJWT (auth stateless) + Doctrine ORM
    ■
    ▼ TCP:5432 (PDO)
TIER 3 – Base de données
  PostgreSQL 16 (Supabase en production)
  8 tables, clés étrangères, index, contraintes
```

6.3 Pattern MVC appliqué

Couche MVC	Implémentation Symfony	Responsabilité
Modèle	src/Entity/ + src/Repository/ + src/Service/	Entités Doctrine + accès données + logique métier

Couche MVC	Implémentation Symfony	Responsabilité
Vue	Application React (frontend)	SPA — aucun template Twig côté Symfony
Contrôleur	src/Controller/ (/api/*)	Validation requête HTTP, sérialisation JSON

■ La logique métier est entièrement encapsulée dans les Services. Les Controllers se limitent à valider la requête HTTP et sérialiser la réponse JSON. Les Repositories isolent l'accès aux données — aucun SQL direct dans les Controllers.

6.4 Endpoints API REST (vue d'ensemble)

Controller	Nb endpoints	Préfixe	Rôles min. requis
AuthController	3	/api/login, /token/refresh, /logout	Public / Tous
BibliothequeController	6	/api/bibliotheques	super_admin
CguController	5	/api/cgu	Public (lecture) / super_admin (écriture)
UtilisateurController	8	/api/utilisateurs	admin / self
LivreController	6	/api/livres	Tous (lecture) / bibliothecaire (écriture)
ExemplaireController	6	/api/exemplaires	Tous (lecture) / bibliothecaire (écriture)
PretController	6	/api/prets	bibliothecaire / adherent (lecture propre)
DemandeMigrationController	5	/api/migrations	adherent (créer) / admin (valider)
StatController	2	/api/stats	admin / super_admin
Total	51	—	—

6.5 Authentication JWT — flux complet

[illegible]

Payload JWT étendu :

```
{
  "sub": 42,
  "email": "bibliothecaire@ipssitheque.fr",
  "role": "bibliothecaire",
  "bibliotheque_id": 1,
  "bibliotheque_nom": "IPSSitheque",
  "must_change_password": false,
  "cgu_accepted_version": "1.1",
  "duret_pret_jours": 21,
  "exp": 1746720000
}
```

7. Développement Réalisé

7.1 Backend Symfony — vue d'ensemble

Composant	Statut	Détail
9 Controllers	■	51 endpoints REST couvrant l'ensemble des besoins fonctionnels
8 Entités Doctrine	■	Mappées sur 8 tables PostgreSQL via Doctrine ORM
5 Migrations	■	Schéma évolutif de l'initialisation aux ajouts J5/J6
2 Services métier	■	PretService (logique prêts) + GoogleBooksService (API externe)
1 EventSubscriber	■	JwtCreatedSubscriber — payload étendu (rôle, bibliothèque_id...)
Rate Limiter	■	10 req/5min par IP + verrou applicatif 5 tentatives/15 min
DataFixtures	■	Données de test pour CI et développement local

7.2 Entités Doctrine — champs notables

Entité	Champs clés ajoutés en cours de projet
Bibliotheque	duret_pret_jours (durée de prêt configurable, défaut 21j)
Utilisateur	must_change_password, cgu_accepted_version, prets_suspendus, login_attempts, locked_until
Exemplaire	Constante STATUT_PERDU
Pret	STATUT_PERDU, etat_depart, etat_retour
CguVersion	version, contenu (TEXT brut, paragraphes séparés par \n), date_effet, date_publication, publie_par
RefreshToken	token (hex 64 chars), valid_until, utilisateur_id

7.3 Frontend React — pages implémentées

Page / Composant	Statut	Rôles	Fonctionnalités principales
LoginPage	■ Complet	Tous	Auth JWT, brute-force (429)
PremierLoginMotDePassePage	■ Complet	Tous	Changement mdp obligatoire
PremierLoginCguPage	■ Complet	Tous	Acceptation CGU dynamique
DashboardPage	■ Complet	Staff + adhérent	KPIs, prêts en retard
CataloguePage	■ Complet	Tous	CRUD livres, ISBN, exemplaires
PretsPage	■ Complet	Staff	Prêts, retours, "PERDU"

Page / Composant	Statut	Rôles	Fonctionnalités principales
AdherentsPage	■ Complet	Admin+	CRUD utilisateurs, suspensions
BibliothequeManagementPage	■ Complet	Admin+	Config bibliothèque
DemandeMigrationPage	■ Complet	Tous	Demandes migration
MonComptePage	■ Complet	Tous	6 onglets : profil, sécurité, RGPD, CGU...

7.4 Services frontend

9 services encapsulent tous les appels API : authService, utilisateurService, livreService, exemplaireService, pretService, bibliothequeService, demandesMigrationService, cguService, statsService. L'intercepteur Axios gère automatiquement l'injection du JWT, le refresh silencieux des tokens expirés, et la déconnexion en cas d'échec de refresh.

8. Fonctionnalités Avancées

8.1 Système de premier login

Lors de la création d'un compte par un admin, le flag `must_change_password = true` est positionné. Au premier login, le JWT contient ce flag et le `ProtectedRoute` intercepte toute navigation :

- 1. Redirection forcée vers `/premier-login/mot-de-passe`
- 2. Changement de mot de passe (8 car. min, majuscule, chiffre, caractère spécial)
- 3. Redirection vers `/premier-login/cgu`
- 4. Lecture et acceptation des CGU en vigueur (version dynamique depuis la BDD)
- 5. `cgu_accepted_version` mis à jour via `POST /api/cgu/accept`
- 6. Accès normal à l'application

8.2 Protection anti-brute force

Niveau	Mécanisme	Paramètres
Applicatif (Symfony)	Verrou compte après N tentatives	5 tentatives → verrou 15 minutes
Infrastructure (Symfony)	Rate Limiter par IP sur <code>/api/login</code>	10 tentatives / 5 minutes
Frontend (React)	Message explicite HTTP 429	Affiche durée de déverrouillage restante

8.3 Versionnement des CGU — CMS intégré

BiblioTech dispose d'un système complet de gestion des CGU **sans intervention sur le code** :

- CGU stockées en BDD dans `cgu_versions` (texte brut, paragraphes séparés par retour à la ligne `\n`)
- Le `super_admin` publie une nouvelle version via l'onglet **Gestion CGU** de `MonComptePage`
- Prise d'effet **15 jours après publication** (préavis obligatoire)
- Bannière de notification affichée à tous les utilisateurs pendant le préavis
- À la date d'effet, les utilisateurs non à jour sont redirigés vers la page d'acceptation

8.4 Gestion des prêts perdus

Un bibliothécaire peut déclarer un exemplaire perdu via `PATCH /api/exemplaires/{id}/perdu`. L'exemplaire passe au statut perdu, le prêt associé également. Côté frontend, la colonne « Date de rendu » affiche **PERDU** au lieu d'une date.

8.5 Durée de prêt configurable

Chaque bibliothèque définit sa propre durée de prêt standard (champ `duret_pret_jours`, défaut : 21 jours). `PretService` utilise cette valeur pour calculer `date_retour_prevue`. La valeur est incluse dans le payload JWT pour

éviter un appel API supplémentaire à chaque création de prêt.

9. Sécurité

9.1 OWASP Top 10 — couverture complète

Menace OWASP	Contre-mesure implémentée
A01 — Broken Access Control	RBAC strict (AttributeVoter Symfony) + isolation bibliotheque_id sur tous les endpoints
A02 — Cryptographic Failures	Argon2id (mots de passe), RS256 4096-bit (JWT), HTTPS en production
A03 — Injection SQL	Doctrine ORM + requêtes préparées uniquement. Aucun SQL dynamique sans paramètres
A04 — Insecure Design	Architecture stateless JWT, séparation frontend/backend, validation des DTOs
A05 — Security Misconfiguration	Variables sensibles dans .env.local (non versionné), clés JWT hors du code source
A06 — Vulnerable Components	Dépendances vérifiées via composer audit et npm audit avant chaque merge
A07 — Auth Failures	Rate Limiter Symfony sur /api/login + verrou applicatif 15 min après 5 tentatives
A08 — Data Integrity Failures	Validation des entrées via Symfony Validator sur tous les DTOs
A09 — Logging Failures	Monolog configuré pour logger les tentatives échouées et accès refusés
A10 — SSRF	Seul appel externe (Google Books) via GoogleBooksService — URL non paramétrable par l'utilisateur

9.2 Contrôle d'accès RBAC

Rôle	Périmètre d'accès	Isolation
super_admin	Tout — gestion globale de la plateforme	Aucune (accès global)
admin	Sa bibliothèque + gestion des comptes	bibliotheque_id du token
bibliothecaire	Catalogue + prêts de sa bibliothèque	bibliotheque_id du token
adherent	Lecture catalogue + ses propres prêts	bibliotheque_id + sub du token

9.3 Audit de sécurité Jalon 6 — 14 vulnérabilités corrigées

Code	Sévérité	Description
CVE-B01	Critique	Crash PHP sur PretController::retour() — variable \$request non injectée
CVE-B02	Haute	Exposition de toutes les bibliothèques à tout utilisateur authentifié
CVE-B03	Haute	Élévation de privilège — admin pouvait créer super_admin
CVE-B04	Haute	Validation de force du mot de passe absente sur 2 endpoints
CVE-B05	Haute	Absence de validation de version CGU côté serveur

Code	Sévérité	Description
CVE-B06	Haute	Isolation multi-tenant absente sur prêts (create + retour)
CVE-B08	Moyenne	Rate Limiter configuré mais non connecté à AuthController
CVE-B09	Haute	4 méthodes UtilisateurController sans contrôle bibliotheque_id
CVE-B10	Moyenne	Validation mdp absente dans update() et create()
CVE-B11	Moyenne	Endpoint byEmail accessible aux adhérents (exposition RGPD)
CVE-B12	Moyenne	Statut exemplaire non validé — corruption d'état métier possible
CVE-B13	Moyenne	Isolation multi-tenant absente sur PretController::show() et byAdherent()
CVE-B14	Haute	Auto-modification mdp sans vérification du mot de passe actuel
BUG-01	—	Champ mot_de_passe vs password dans MonComptePage (MDP ignoré silencieusement)

■ Toutes les 14 vulnérabilités ont été corrigées avant le livrable final. Voir [conception/SECURITY_NOTE.md](#) pour le détail complet des correctifs.

9.4 Risques résiduels documentés (acceptés)

Risque	Justification	Amélioration future
must_change_password enforcement côté frontend uniquement	SPA React — accès uniquement depuis l'interface authentifiée. ProtectedRoute vérifie à chaque navigation.	Kernel event listener Symfony (backend enforcement)
JWT non invalidé sur logout	Pattern stateless standard JWT. Le refresh token (longue session) est bien invalidé. TTL access token : 8h.	Token blacklist Redis
JWT dans localStorage	Risque faible — SPA sans contenu tiers dynamique injecté.	Migration vers cookies HttpOnly + SameSite=Strict

10. Tests

10.1 Stratégie de tests

Type	Outil	Périmètre	Résultats
Tests unitaires	PHPUnit 11	PretService, GoogleBooksService	14 tests
Tests d'intégration	Symfony WebTestCase	AuthController, LivreController, PretController, ExempleController, BibliothequeController, UtilisateurController	26 tests
Total	—	—	40 tests, 127 assertions

■ Les tests d'intégration utilisent une base PostgreSQL dédiée réinitialisée avant chaque suite via DataFixtures Doctrine (1 bibliothèque, 1 admin, 1 bibliothécaire, 3 adhérents, 5 livres, 8 exemplaires). La CI GitHub Actions exécute ces tests automatiquement à chaque push.

10.2 Résultats PHPUnit

PHPUnit 11.x

```
Unit/Service/PretServiceTest ..... 8 tests OK
Unit/Service/GoogleBooksServiceTest ..... 6 tests OK
Integration/Controller/AuthControllerTest . 8 tests OK
Integration/Controller/LivreControllerTest 9 tests OK (isolat. tenant)
Integration/Controller/PretControllerTest . 9 tests OK
```

Tests: 40, Assertions: 127, OK (40 tests, 127 assertions)

10.3 Cas de tests représentatifs

PretService (unitaires) :

Test	Résultat attendu
testEnregistrerPretExemplaireDisponible	Prêt créé, statut exemplaire → emprunte
testEnregistrerPretExemplaireIndisponible	Exception ExemplaireUnavailableException
testEnregistrerPretMaxPrets (5 prêts en cours)	Exception PretMaxReachedException
testEnregistrerRetourPretDejaRendu	Exception PretAlreadyReturnedException
testCalculDateRetourDefault	Date = J+21 (duree_pret_jours)

AuthController (intégration) :

Test	Résultat attendu
testLoginSucces	200 + token JWT dans la réponse

Test	Résultat attendu
testLoginMauvaisMotDePasse	401 + code AUTH_INVALID_CREDENTIALS
testLoginCompteDesactive	403 + code AUTH_ACCOUNT_DISABLED
testLoginRateLimiter (6 tentatives)	429 après la 5ème tentative
testAccesSansToken	401
testAccesTokenExpire	401 + code AUTH_TOKEN_EXPIRED

LivreController (intégration — isolation tenant) :

Test	Rôle	Résultat attendu
testPostLivreAdherentInterdit	adherent	403 — création interdite
testPostLivreIsbnDuplique	bibliothecaire	409 — LIVRE_ISBN_DUPLICATE
testDeleteLivreBibliothecaireInterdit	bibliothecaire	403 — suppression admin only
testGetLivreAutreBibliotheque	admin autre bibliothèque	403 — isolation tenant

11. Infrastructure & CI/CD

11.1 Architecture Docker

Service	Image	Port	Rôle
app	PHP 8.3-FPM + Nginx (Alpine)	8080	API REST Symfony
db	PostgreSQL 16 Alpine	5432	Base de données locale (développement)
frontend	Node 20 Alpine	5173	Dev server React (hot-reload)

■ ■ En production, le service db est remplacé par Supabase (PostgreSQL cloud managé). Le backend est déployé sur Render (Web Service Docker). Le frontend est déployé en tant que Static Site (npm run build → dist/).

11.2 Pipeline CI/CD — GitHub Actions

Job	Déclencheur	Étapes
backend-tests	Tout push / PR (main, develop)	PHP 8.3, composer install, génération clés JWT RSA 4096-bit, migrations, fixtures, PHPUnit --testdox
frontend-lint	Tout push / PR (main, develop)	Node 20, npm ci, tsc --noEmit (vérification des types TypeScript)
docker-build	Tag v*.*.* uniquement (release)	Docker Buildx, login Docker Hub, build & push images backend + frontend (:latest + :vX.Y.Z)

11.3 Variables d'environnement de production

Variable	Usage	Obligatoire
DATABASE_URL	DSN PostgreSQL (Supabase)	■
JWT_SECRET_KEY	Chemin clé privée RSA	■
JWT_PUBLIC_KEY	Chemin clé publique RSA	■
JWT_PASSPHRASE	Passphrase clé RSA (vide si clés sans passphrase)	■
APP_SECRET	Secret Symfony (32 caractères aléatoires)	■
APP_ENV	prod	■
CORS_ALLOW_ORIGIN	URL exacte du frontend (ex: https://bibliotech.onrender.com)	■
GOOGLE_BOOKS_API_KEY	Clé API Google Books (fallback saisie manuelle si absent)	■ ■ Partiel

11.4 Migrations Doctrine

Fichier	Contenu
Version20260501000000	Schéma initial — 6 tables, clés étrangères, index
Version20260526000000	Table refresh_tokens
Version20260601000000	Champs must_change_password, login_attempts, locked_until sur utilisateurs
Version20260602000000	Champs cgu_accepted_version, prets_suspendus, duret_pret_jours
Version20260603000000	Table cgu_versions, champs etat_depart/etat_retour sur prets

En production (Render), les migrations sont exécutées **automatiquement au démarrage du conteneur** via `docker/entrypoint.sh` (`doctrine:migrations:migrate --no-interaction`). La commande est non-bloquante : si la base est temporairement inaccessible au boot, un avertissement est loggué et le serveur démarre quand même.

12. Conformité RGPD

12.1 Registre des traitements (Article 30)

Traitement	Finalité	Durée de conservation
Gestion des comptes	Authentification et accès aux services	Durée de vie du compte + 1 an (anonymisation)
Gestion des prêts	Suivi des emprunts et historique	5 ans (obligation comptable)
Logs d'authentification	Sécurité — détection des tentatives d'intrusion	90 jours glissants
Acceptation des CGU	Preuve légale de consentement	Durée de vie du compte + 1 an

12.2 Droits des utilisateurs implémentés

Droit RGPD	Implémentation
Droit d'accès	GET /api/utilisateurs/me — consultation des données personnelles
Droit de rectification	PATCH /api/utilisateurs/me — modification nom, prénom, email, mot de passe
Droit à l'effacement	DELETE /api/utilisateurs/{id} — anonymisation RGPD : données remplacées par [SUPPRIMÉ]
Consentement CGU	POST /api/cgu/accept — enregistrement de l'acceptation avec version et date

12.3 Mesures techniques

- Mots de passe hachés **Argon2id** (memory_cost: 65536, time_cost: 4)
- Communications en **HTTPS** en production
- Clés JWT RSA **hors du dépôt Git** (.gitignore)
- Variables sensibles en **variables d'environnement** (jamais en dur dans le code)
- **Minimisation** : seules les données nécessaires sont stockées (pas de date de naissance, pas de téléphone obligatoire)

13. Bilan Global & Perspectives

13.1 Ce qui a été réalisé

Domaine	Réalisé	Statut
Architecture	API REST Symfony 7 + SPA React 19 (Vite + TypeScript)	■
Base de données	PostgreSQL 16, Doctrine ORM, 8 entités, 5 migrations	■
Authentification	JWT RS256 + refresh token rotation (8h/30j)	■
RBAC	4 rôles (super_admin, admin, bibliothécaire, adherent)	■
Multi-tenant	Isolation complète par bibliotheque_id sur tous les endpoints	■
Sécurité	14 vulnérabilités corrigées, OWASP Top 10 couvert	■
API externe	Google Books API (ISBN → couverture + métadonnées)	■
RGPD	Anonymisation sur suppression, registre Article 30, CGU versionnées	■
Docker	docker-compose.yml complet, Dockerfiles backend et frontend	■
CI/CD	Pipeline GitHub Actions (tests + build Docker sur release)	■
Tests	PHPUnit intégration + unitaire (8 suites, 40 tests, 127 assertions)	■
Déploiement	Render (backend Docker) + Static Site (frontend) + Supabase	■

13.2 Difficultés techniques surmontées

- **Isolation multi-tenant** : pattern récurrent — vérification bibliotheque_id manquante sur de nombreux endpoints découverts lors de l'audit J6. Résolu par un audit systématique contrôleur par contrôleur.
- **JWT stateless vs gardes applicatives** : la logique must_change_password ne pouvait pas être dans le token (stale après émission). Solution : vérification sur chaque route protégée via ProtectedRoute.
- **Rate Limiter Symfony 7** : l'injection #[Target('loginIp')] nécessite une syntaxe spécifique non documentée dans les exemples officiels de LexikJWT.
- **Docker Compose chemins relatifs** : les chemins ./backend depuis dev/docker/ étaient incorrects. Corrigés en ../backend.
- **Passphrase JWT en production** : les clés générées au build Docker sans passphrase nécessitent JWT_PASSPHRASE="" dans les variables d'environnement Render.
- **IPv6 / Supabase pooler** : Render Free n'a pas d'IPv6 sortant. La connexion directe Supabase (port 5432 direct) résout en IPv6 → timeout. Solution : utiliser l'URL pooler Supabase (aws-1-eu-west-2.pooler.supabase.com, port 5432, mode Session).
- **Permissions cache Symfony** : le répertoire var/cache/prod/ n'existait pas au démarrage → *Permission denied* sur toute requête. Résolu dans entypoint.sh : cache:warmup pré-crée tous les sous-dossiers puis chmod -R 777 var/.
- **Pools cache Doctrine manquants** : doctrine.yaml (prod) référence doctrine.system_cache_pool et doctrine.result_cache_pool sans qu'ils soient déclarés. Résolu dans config/packages/cache.yaml avec l'adapteur filesystem.

13.3 Conformité CDC technique

Exigence CDC	Implémentation	Statut
Symfony	Symfony 7.3 + PHP 8.3	■
React / Angular SPA	React 19 + Vite + TypeScript	■
PostgreSQL	PostgreSQL 16 via Doctrine ORM	■
API externe	Google Books API (GoogleBooksService)	■
Docker	3 services (app, db, frontend)	■
Git branches	main / develop / feature/*	■
CI/CD	GitHub Actions — 3 jobs	■
PHPUnit (unit + intégration)	8 suites, 40 tests, 127 assertions	■
OWASP Top 10	Documenté + implémenté + audité (14 CVE corrigés)	■
Argon2id / bcrypt	Argon2id memory_cost 65536, time_cost 4	■
RGPD	CGU, registre Article 30, anonymisation, consentement	■
MVC	Architecture Symfony Controllers / Services / Entities	■
Responsive	Layout mobile adapté (MobileHeader, sidebar drawer)	■

13.4 Perspectives d'évolution

Amélioration	Priorité	Description
Tests E2E Playwright	Haute	Scénarios complets navigateur (login → prêt → retour)
Enforcement backend must_change_password	Haute	Kernel event listener Symfony — remplacerait la garde frontend
Réduction TTL JWT	Moyenne	Passer de 8h à 15-30 min + refresh silencieux toutes les 5 min
Notifications email	Moyenne	Relances automatiques pour les prêts en retard (Mailer Symfony)
Cookies HttpOnly	Moyenne	Migration depuis localStorage — renforcement sécurité session
Audit OWASP ZAP automatisé	Faible	Intégration dans la pipeline CI sur l'instance de staging
Application mobile	Faible	L'API REST est déjà consommable par une app React Native

13.5 Conclusion

BiblioTech est une plateforme SaaS multi-tenant fonctionnelle, sécurisée et conforme aux exigences du CDC technique. En 6 jalons couvrant janvier à juillet 2026, le projet a produit une API REST Symfony 7 complète (51 endpoints, 9 controllers, 8 entités), une SPA React 19 avec 10 pages et gestion complète des rôles, un pipeline CI/CD GitHub Actions, et un audit de sécurité ayant identifié et corrigé 14 vulnérabilités.

La stack choisie (Symfony + React + PostgreSQL + Docker) est robuste, maintenable, et pleinement conforme aux exigences du référentiel CDA. Le projet est prêt pour la soutenance et le déploiement en production.

Dossier de projet BiblioTech — Grégory Sergent — Formation CDA — IPSSI Grande École d'Informatique — Juillet 2026